

Khalifa University
Department of Computer Science
Introduction to Software Engineering (COSC 336)
Lab-based Running Project

Fall 2026

Intelligent, AI-Powered Circular Campus Resource Exchange and Asset Life Cycle Management System

Project Description

This semester, the software engineering project will focus on designing and prototyping *an Intelligent, AI-Powered Circular Campus Resource Exchange and Asset Life Cycle Management System*. The project will give you the opportunity to apply the complete software development life cycle such as requirements, design, implementation, testing, and documentation while exploring how emerging AI technologies complement and extend traditional asset-management and resource-exchange solutions.

Universities regularly purchase furniture, computers, electronic devices, laboratory equipment, books, office materials, and other resources while similar usable items remain unused, stored, or discarded by other departments. The main problem is not always the absence of reusable resources. It is often the absence of an integrated system that can identify, classify, match, transfer, maintain, and track those resources throughout their life cycle.

Traditional university asset management systems are usually designed to record ownership, location, purchase information, and disposal status. Although these functions are important, such systems often do not support intelligent internal reuse, semantic matching between available assets and departmental requests, prediction of future resource demand, or measurement of the environmental and financial benefits of circular resource management.

The *Intelligent, AI-Powered Circular Campus Resource Exchange and Asset Life Cycle Management System* should be designed to address these challenges by combining classic software engineering practices with emerging AI-driven enhancements. The purpose of this project is not only to build a

functional prototype, but also to give students practical experience of how software engineering is evolving to support intelligent, sustainable, and data-driven decision-making.

At its foundation, the system will provide the classic functionality expected from an internal campus resource-exchange and asset-management platform:

- Registering and maintaining campus asset records, publishing surplus, underutilized, or unwanted resources.
- Allowing departments to submit resource requests, searching, filtering, reserving, and requesting available resources.
- Managing approvals, transfers, ownership, custody, and location changes, recording inspections, maintenance, repairs, donations, recycling, and disposal actions and managing user accounts with different roles and permissions.

These baseline modules will allow students to practice core software engineering skills, including requirements gathering, system modeling using UML and data-flow diagrams, interface design, modular programming, workflow development, security design, and systematic testing.

In parallel, students will extend the system with modern AI-powered functionalities that reflect the increasing integration of artificial intelligence into enterprise and sustainability-management applications.

Examples include:

- An AI-based resource-matching engine that compares asset listings with departmental requests using semantic similarity, technical compatibility, condition, quantity, location, and urgency.
- An AI-assisted asset-classification function that recommends categories, tags, and standardized descriptions for newly registered resources.
- A sustainability-optimization module that recommends whether an asset should be reused, transferred, repaired, refurbished, donated, recycled, or retired.
- An LLM-powered assistant that allows users to search for resources, submit requests, ask asset-related questions, and navigate system services using natural language.
- Generative AI reporting summarizes resource-exchange activities, maintenance trends, avoided purchases, financial savings, waste diversion, and estimated carbon-emission reductions.

By working on both classic and AI-augmented approaches, students will directly compare two perspectives:

- The traditional systems-engineering perspective, which is structured, deterministic, transparent, and rule-based and
- The modern AI-enhanced perspective, which is adaptive, user-centered, and recommendation-driven.

This dual focus ensures that students gain a strong foundation in conventional software engineering while also developing an understanding of how AI can improve classification, matching, life cycle decision-making, and sustainability management.

Equally important, the project is designed to simulate real-world engineering practice. Students will work through the software life cycle in phases: from planning and requirement gathering and analysis to architecture and design, to development, testing, and documentation. They will apply best practices such

as modular design, user-centered interfaces, secure access control, data validation, agile collaboration, and human oversight of AI recommendations.

At the same time, students will engage in critical reflection by considering questions such as:

- What value does AI add to a circular campus resource-exchange system?
- How can the system explain why a particular asset was recommended?
- How should uncertainty in sustainability estimates be communicated?
- What risks arise when AI influences transfer, repair, donation, recycling, or disposal decisions?
- Which decisions must remain under human control?

Therefore, the project is not only about building an application. It is about learning to think and act like a software engineer by:

- Identifying stakeholders, departmental needs and designing robust and scalable architecture
- Integrating emerging technologies responsibly
- Measuring environmental and economic value and delivering a professional system with technical, organizational, financial, and social value.

Objectives

The objectives of the project are as follows:

1. Develop Classic Software Engineering Functionality (Baseline System)

Students are expected to design and implement the core functions of a circular campus resource-exchange and asset life cycle management system using conventional software engineering approaches. These functions will demonstrate the ability to model requirements, design relational data structures, implement workflow logic, and build reliable user interfaces.

Specifically, students will:

- Implement an asset-registration and inventory-management module through which authorized users can record an asset's name, description, category, quantity, owner, department, location, purchase date, estimated value, condition, availability, photographs, and supporting documents.
- Develop a circular resource marketplace in which departments can publish surplus, underutilized, or unwanted assets and make them visible to authorized campus users.
- Build a departmental resource-request module that allows users to specify the required resource category, purpose, specifications, quantity, preferred condition, urgency, location, and required date.
- Provide structured search and filtering based on category, condition, department, location, quantity, availability, asset age, estimated value, and current life cycle status.
- Implement reservation and request-management functions that allow users to express interest in an available asset and track the status of the request.
- Develop approval workflows for asset release, reservation, transfer, receipt, donation, recycling, and disposal.
- Implement transfer-management functions that record approval, collection or delivery arrangements, custody changes, new location, transfer confirmation, and supporting documents.
- Build an asset life cycle history that records acquisition, allocation, utilization, inspection, maintenance, repair, transfer, refurbishment, donation, recycling, retirement, and disposal events.

- Develop a maintenance and repair module through which authorized users can report defects, request inspection, record repair actions, monitor status, capture costs, and update the asset's condition after service.
- Design user management and authentication with role-based access for department representatives, asset custodians, requesters, administrators, procurement officers, finance officers, maintenance staff, sustainability officers, and system administrators.
- Generate conventional operational reports covering available assets, open requests, completed transfers, maintenance history, ownership changes, approvals, and asset status.

These functions represent the classic perspective of software engineering: structured design, rule-based logic, data management, transparent workflows, and a clear separation of responsibilities between system modules.

2. Integrate Modern AI-Enhanced Functionalities (Extended System)

In parallel with the baseline system, students will explore how AI technologies can extend traditional asset-management designs and provide more intelligent, user-centered, and sustainability-oriented services. This reflects the contemporary perspective of software engineering, where AI is increasingly embedded in enterprise applications.

Students will:

- Design and integrate an AI-assisted asset-classification service that analyzes titles, descriptions, specifications, images, and documents to recommend a category, subcategory, standardized tags, possible material type, and missing metadata.
- Develop an AI-based semantic resource-matching engine that compares available asset listings with departmental requests based on meaning rather than exact keywords.
- Configure the matching engine to consider category compatibility, functional purpose, technical specifications, quantity, asset condition, requested condition, physical location, urgency, required date, transfer cost, and repair cost.
- Generate a compatibility score for each recommended asset-request combination and provide a human-readable explanation of the main factors that influenced the recommendation.
- Develop a prioritization model that ranks suitable matches according to functional suitability, urgency, transfer cost, repair requirements, waste-diversion benefit, and estimated carbon-emission reduction.
- Build a sustainable-action recommendation module that recommends whether an asset should be retained, transferred, repaired, refurbished, donated, disassembled for usable components, recycled, or disposed of as a final option.
- Design an LLM-powered circular campus assistant that supports natural-language interaction for asset search, request submission, asset-history questions, status tracking, policy guidance, sustainability questions, and navigation of system functions.
- Apply generative AI methods to transform operational data into summarized monthly reports, sustainability updates, management bulletins, high-demand-category summaries, maintenance insights, and lists of recommended actions.

By working on these enhancements, students will critically reflect on the differences between traditional systems and AI-augmented systems, identifying both the added value and the limitations or risks of AI

integration, including accuracy, explainability, bias, data quality, privacy, uncertainty, security, and ethical concerns.

3. Compare and Reflect on Classic vs. AI-Enhanced Solutions

An important objective of this project is for students to compare the two perspectives.

- Classic systems are rule-driven, transparent, predictable, and deterministic, but may be rigid and dependent on exact data structures or keywords.
- AI-driven systems are adaptive, semantic, user-friendly, and powerful, but may be uncertain, opaque, biased, or unpredictable.

For example:

- A classic search function may return assets only when an exact keyword or category is selected, while an AI semantic-search function may identify functionally similar assets described using different terminology.
- A classic matching rule may check category, quantity, and availability, while an AI-based matching model may rank alternatives using specifications, condition, urgency, location, cost, and sustainability impact.
- A classic report may display fixed totals and tables, while a generative AI report may summarize trends, explain anomalies, and suggest management actions.

Students will be required to discuss these differences explicitly in their documentation. They should explain where deterministic rules are more appropriate, where AI provides meaningful added value, and how human oversight can reduce the risks of AI-supported decision-making. This comparison should demonstrate not only technical competence, but also critical thinking about modern software engineering, sustainability optimization, and responsible AI integration.

4. Apply Modern Software Engineering Practices

Beyond functional requirements, students are expected to apply professional software engineering practices to ensure that the project is maintainable, testable, secure, usable, scalable, and academically rigorous. This includes:

- Requirement Analysis: producing detailed functional and non-functional requirements, stakeholder maps, business rules, use cases, data requirements, AI requirements, and sustainability-measurement requirements.
- Architectural Design: selecting and documenting a modular, layered, service-oriented, or microservices-based architecture that emphasizes separation of concerns, maintainability, integration, scalability, and security.
- Modeling: defining models for users, departments, assets, categories, requests, matches, approvals, transfers, maintenance records, sustainability impacts, notifications, and audit logs.
- AI Integration Design: defining model inputs, outputs, confidence information, explanation requirements, approval controls, version tracking, fallback behavior, and the boundary between automated recommendations and human decisions.

- User-Centered Design – creating intuitive interfaces, dashboard layouts, user journeys, mockups, accessibility considerations, and usability requirements for different campus roles.
- Security and Privacy – applying authentication, authorization, data minimization, secure storage, controlled access to financial and operational information, audit trails, and protection against unauthorized changes.
- Testing and Quality Assurance: developing and executing unit, integration, system, workflow, usability, security, performance, database, AI-quality, explainability, and user-acceptance tests.
- AI Evaluation: evaluating classification quality, match relevance, recommendation acceptance, fairness, and the usefulness of generated explanations.
- Documentation: providing clear documentation at every phase, explaining design choices, development trade-offs, assumptions, limitations, risks, testing outcomes, and opportunities for future improvement.

5. Develop Transferable Skills

Through this project, students will strengthen:

- Technical skills in requirements engineering, system architecture, interface design, software implementation, and testing.
- Practical AI integration skills, especially in semantic matching, classification, recommendation systems, generative reporting, and LLM-based conversational interfaces.
- Sustainability-analysis skills, including the definition and interpretation of waste-diversion, reuse, repair, financial-saving, and carbon-reduction indicators.
- Teamwork and collaboration skills by distributing tasks, coordinating modules, reviewing work, managing versions, and integrating system components.
- Communication and reporting skills through professional technical documentation, project demonstrations, presentations, user guides, and oral defense of design decisions.
- Critical reflection by analyzing the role of AI in resource allocation and asset management, comparing AI recommendations with traditional rule-based decisions, and evaluating social, ethical, organizational, and environmental consequences.
- Problem-solving and decision-making skills by balancing functional suitability, cost, operational urgency, asset condition, fairness, safety, and sustainability impact.

Learning Outcome

By the end of this project, students should be able to demonstrate that they can:

- Engineer a functioning software prototype containing both classic and AI-enhanced modules.
- Design and implement an internal circular campus resource marketplace for available assets and departmental requests.
- Model and track an asset throughout its life cycle, from registration and allocation to maintenance, transfer, donation, recycling, or retirement.
- Apply AI methods to classify resources, generate semantic matches, prioritize alternatives, and recommend sustainable actions.
- Apply modern software engineering practices to manage complexity, scalability, security, maintainability, usability, and integration.
- Define and calculate sustainability indicators such as waste diversion, asset-life extension, reuse rate, financial savings, and estimated carbon-emission reductions.

- Evaluate the quality, usefulness, uncertainty, explainability, fairness, and limitations of AI-generated recommendations.
- Compare structured, deterministic software functions with adaptive, probabilistic AI-enhanced functions.
- Design human-in-the-loop workflows in which AI supports decisions while authorized users remain accountable for approvals and high-impact actions.
- Present the solution in a structured, professional, and academically rigorous manner.

This dual-perspective project ensures that students leave the course with foundational skills in classic software engineering while also gaining practical insight into the future of intelligent enterprise systems, circular resource management, and AI-supported sustainability optimization. AI determines how campus resources should be matched, allocated, and managed; sustainability defines what the AI should optimize.

Project Phases

• Phase 1: Planning and Requirement Gathering

- Define the project problem, scope, objectives, assumptions, and expected benefits.
- Identify stakeholders such as department representatives, asset custodians, requesters, administrators, etc.
- Map stakeholders to specific system features, data needs, permissions, responsibilities, and approval decisions.
- Study the current process for identifying surplus assets, requesting resources, approving transfers, recording maintenance, and managing donation, recycling, or disposal.
- Define measurable project success criteria, including functional performance, user acceptance, match usefulness, avoided purchases, reuse, repair, and sustainability outcomes.
- Develop a preliminary project plan with timelines, milestones, task assignments, dependencies, risks, and resource allocations.

• Phase 2: Feasibility Study

- Assess the technical feasibility of the application architecture, database, AI services, semantic search, reporting, and system integrations.
- Assess financial feasibility by considering development resources, infrastructure requirements, potential savings from avoided purchases, and long-term maintenance costs.
- Assess operational feasibility by evaluating whether departments, asset custodians, administrators, maintenance teams, procurement staff, and sustainability officers can adopt the proposed workflows.
- Assess schedule feasibility in relation to the semester timeline and the availability of data, tools, team members, and technical resources.
- Assess data and AI feasibility, including the availability, completeness, quality, consistency, and privacy of asset, request, maintenance, transfer, and procurement data.
- Identify legal, ethical, privacy, security, procurement, ownership, safety, and university-policy constraints.

• Phase 3: Requirement Analysis

- Develop a detailed set of functional and non-functional requirements. Define the actors, roles, permissions, business rules, approval stages, asset states, request states, transfer states, and maintenance states.
- Define requirements for asset registration, resource publication, request submission, search, matching, reservation, approval, transfer, maintenance, donation, recycling, reporting, notifications, and auditability.
- Define AI requirements for classification, semantic matching, ranking, sustainable-action recommendations, explanations, confidence information, user overrides, and model-version tracking.
- Define sustainability requirements for waste diversion, avoided purchases, asset-life extension, reuse, repair, financial savings, and estimated carbon-emission reductions. Create use-case diagrams and detailed use-case descriptions. Prioritize requirements into essential prototype functions, desirable extensions, and possible future enhancements.

• Phase 4: Architecture Design

- Design the system architecture, including the integration of classic application modules and modern AI-enhanced functionalities.
- Define presentation, application, AI and analytics, data, integration, notification, security, and reporting components. Determine how the application will communicate with the classification service, semantic-matching engine, ranking model, recommendation engine, and LLM assistant.
- Define human approval boundaries and fallback behavior when AI services are unavailable or produce low-confidence outputs. Develop mockups for asset registration, request submission, recommendation review, transfer approval, maintenance tracking, sustainability dashboards, reports, and conversational interaction.
- Define the data models and prepare context-level and detailed data-flow diagrams. Document security boundaries, access-control rules, audit logging, data storage, backup, recovery, and integration points.

• Phase 5: Design and Modeling

Create detailed designs using UML diagrams, including:

- use-case diagrams, class diagrams, sequence diagrams, activity diagrams, asset state diagrams
- resource-request state diagrams transfer and approval state diagrams.
- component diagrams, deployment diagrams and AI recommendation workflow diagrams.

Develop an entity-relationship diagram covering users, roles, departments, assets, categories, requests, recommendations, reservations, approvals, transfers, inspections, maintenance records, sustainability impacts, notifications, model versions, and audit logs. Define interface contracts between software modules and AI services. Design the recommendation score, explanation structure, sustainability calculations, and model-monitoring records.

• Phase 6: Development

Implement a working prototype. The prototype should include, at minimum:

- User registration or administrator-created accounts, authentication and role-based access control;
- Department management, asset registration and status management.
- Publication of available or surplus resources, resource-request submission and tracking. Search and filtering. AI-assisted classification or tagging, AI-based resource matching and recommendation ranking, recommendation explanation and user review.
- Transfer approval and confirmation, ownership, maintenance or repair tracking.
- Sustainability indicators and dashboard summaries, notifications and basic operational and AI-generated reports.

• Phase 7: Testing and Quality Assurance

Perform comprehensive testing of the system's classic and AI-enhanced functionalities. Testing should include:

- unit testing of individual functions and modules;
- integration testing between application services;
- system testing end-to-end workflows;
- AI classification evaluation, sustainability-calculation validation; and
- user-acceptance testing.

Students should explicitly report the limitations of the test data and avoid claiming model accuracy beyond what the available evidence supports.

• Phase 8: Final Submission with Documentation

Submit the final source code, configuration files, model or service integration files, and project documentation. Provide comprehensive documentation covering the planning, feasibility, requirements, architecture, design, development, testing, deployment, and evaluation phases.

Deliverables

- Project Plan
- Functional and Non-Functional Requirements Document, Use-Case Diagram and Detailed Use-Case Descriptions
- Design Documentation, including Architecture, Mockups, Data-Flow Diagrams, and UML Diagrams, AI Design and Evaluation Documentation
- Developed Classic Functionalities, Developed AI-Enhanced Functionalities, Working Prototype
- Source Code, Testing Reports and Quality Assurance Documentation
- Final Technical Report, Presentation and System Demonstration

This project must be developed using Git and hosted on GitHub. Each team is required to create a GitHub repository for the project and add the instructor and the lab engineer's GitHub usernames as

collaborators by September 15. Development progress must be tracked through regular, meaningful commits throughout the project period. **Submitting the entire final project as a single commit does not satisfy this requirement and will result in a grade penalty.**

The commit history will be used to verify the timeline of work and individual contribution and will be reviewed as part of grading. Repositories with insufficient or irregular commit activity, or with no clear individual authorship in team projects, will be flagged for review.

By following these project phases and deliverables, the project aims to create a modern, user-friendly, secure, and AI-powered circular campus system that integrates core asset-management functionality with intelligent matching, predictive life cycle analysis, and measurable sustainability management.

Time Schedule

While dates may change, here is the tentative schedule:

Phase	Submission deadlines	Deliverable	Weight
Phase 1	In Labs of week starting September 20, 2026.	Initial Plan and Requirement Gathering Document	10%
Phase 2	In Labs of week starting October 4, 2026.	Feasibility Document	10%
Phase 3	In Labs of week starting October 18, 2026.	Requirements Document	10%
Phases 4 and 5	In Labs of week starting November 1, 2026.	Design Document (Architecture and Detailed Design)	20%
Phase 6	In Labs of week starting November 22, 2026	Draft Implementation with Major Features	20%
Phase 7	In Labs of week starting December 6, 2026	Details of testcases for project	10%
Phase 8	In Labs of week starting December 6, 2026	Final submission of project/presentation	20%

Appendix

Introduction

Any software development project must begin with requirements identification. No other task (design, coding, testing) could start until the client's needs had been precisely established and confirmed. Business analysts that are knowledgeable about the topic typically consult with clients to determine which features should be deployed.

This exercise will teach us how to separate functional needs from non-functional requirements based on a problem statement. The main elements of a software requirements specification are functional and non-functional needs.

During the labs you will be able to:

- Recognize ambiguities, inconsistencies, and incompleteness from a requirements specification after finishing this experiment.

- List and describe the functional requirements.
- Describe non-functional requirements.

Requirements

A requirement, according to Sommerville, is a description of what must be carried out. Requirements define the behavior that the target system must exhibit. It outlines what must be done but not how. Requirements engineering is the process of comprehending what a client anticipates from the system that will be constructed and documenting those expectations in a consistent, clear, and intelligible manner. For the system's subsequent design, implementation, and verification, this documentation will be used as a guide. Prior to beginning the planning, design, and implementation of the software system for our customer, it is critical and vital that we have a thorough understanding of its needs. There would be major issues and client unhappiness if we had a clear perspective of what needs to be created and what features are anticipated.

Specifications of Requirements

Any new system that is being designed should have the following three characteristics in its requirements:

- **Clarity:** It should be clear what a system being designed is supposed to do. Consider, for instance, that you are creating a web application for a client. The client demands that a sufficient number of users be able to access the application at once. What is "enough people"? That might be 10 to you, but the client might see it as 100. There is a haziness.
- **Consistency:** The specifications should be consistent.
- **Completeness:** A specific need for a system should outline both the things that the system should and should not be able to achieve. Think of a piece of software being created for an ATM, for instance. The ATM should display an error notice and not dispense any cash if a user enters a value that is higher than the maximum withdrawal amount allowed.

Definition of Requirements Categories

Based on the target audience or subject matter, requirements can be classified into different types, as stated below:

- **User requirements:** They are written in natural language so that both customers can verify their requirements have been correctly identified
- **System requirements:** They are written involving technical terms and/or specifications, and are meant for the development or testing teams

Requirements can be classified into two groups based on what they describe:

- **Functional requirements (FRs):** These describe the functionality of a system -- how a system should react to a particular set of inputs and what should be the corresponding output.
- **Non-functional requirements (NFRs):** They are not directly related what functionalities are expected from the system. However, NFRs could typically define how the system should behave under certain situations. For example, an NFR could say that the system should work with 128MB RAM. Under such condition, an NFR could be more critical than a FR.
-

Non-functional requirements could be further classified into different types like:

- **Product requirements:** For example, a specification that the web application should use only plain HTML, and no frames

- **Performance requirements:** For example, the system should remain available 24x7
- **Organizational requirements:** The development process should comply to SEI CMM level 4

Functional Requirements

Identifying Functional Requirements

Given a problem statement, the functional requirements could be identified by focusing on the following points:

- Identify the high-level functional requirements simply from the conceptual understanding of the problem. For example, a Library Management System, apart from anything else, should be able to issue and return books.
- Identify the cases where an end user gets some meaningful work done by using the system. For example, in a digital library a user might use the "Search Book" functionality to obtain information about the books of his interest.
- If we consider the system as a black box, there would be some inputs to it, and some output in return. This black box defines the functionalities of the system. For example, to search for a book, user gives title of the book as input and get the book details and location as the output.
- Any high-level requirement identified could have different sub-requirements. For example, "Issue Book" module could behave differently for different class of users, or for a particular user who has issued the book thrice consecutively.

Preparing Software Requirements Specifications (SRS)

Once all possible FRs and non-FRs have been identified, which are complete, consistent, and non-ambiguous, the Software Requirements Specification (SRS) is to be prepared. IEEE provides a template available in a separate document, which could be used for this purpose. The SRS is prepared by the service provider and verified by its client. This document serves as a legal agreement between the client and the service provider. Once the concerned system has been developed and deployed, and a proposed feature was not found to be present in the system, the client can point this out from the SRS. Also, if after delivery, the client says a new feature is required, which was not mentioned in the SRS, the service provider can again point to the SRS. The scope of the current experiment, however, doesn't cover writing an SRS.

Use the SRS document (below) to just fill the FR and NFR parts and submit them with the name and id of your group. Collaboration with other groups is not permitted.

Software Requirements Specification

for

<Project>

Version 1.0 approved

Prepared by <authors>

<organization>

<date created>

Table of Contents

Table of Contents	xiv
Revision History	xiv
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References.....	1
2. Overall Description	1
2.1 Product Perspective.....	1
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment.....	2
2.5 Design and Implementation Constraints	2
2.6 User Documentation	2
2.7 Assumptions and Dependencies	2
3. External Interface Requirements	3
3.1 User Interfaces	3
3.2 Hardware Interfaces	3
3.3 Software Interfaces	3
3.4 Communications Interfaces	3
4. System Features	3
4.1 System Feature 1.....	3
4.2 System Feature 2 (and so on).....	4
5. Other Nonfunctional Requirements.....	4
5.1 Performance Requirements	4
5.2 Safety Requirements	4
5.3 Security Requirements	4
5.4 Software Quality Attributes	5
5.5 Business Rules	5
6. Other Requirements	5
Appendix A: Glossary.....	5
Appendix B: Analysis Models.....	5
Appendix C: To Be Determined List.....	5

Revision History

Name	Date	Reason For Changes	Version

Introduction

Purpose

<Identify the product whose software requirements are specified in this document, including the revision or release number. Describe the scope of the product that is covered by this SRS, particularly if this SRS describes only part of the system or a single subsystem.>

Document Conventions

<Describe any standards or typographical conventions that were followed when writing this SRS, such as fonts or highlighting that have special significance. For example, state whether priorities for higher-level requirements are assumed to be inherited by detailed requirements, or whether every requirement statement is to have its own priority.>

Intended Audience and Reading Suggestions

<Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this SRS contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type.>

Product Scope

<Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies. If a separate vision and scope document is available, refer to it rather than duplicating its contents here.>

References

<List any other documents or Web addresses to which this SRS refers. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location.>

Overall Description

Product Perspective

<Describe the context and origin of the product being specified in this SRS. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product. If the SRS defines a component of a larger system, relate the requirements of the larger system to the functionality of this software and identify interfaces between the two. A simple diagram that shows the major components of the overall system, subsystem interconnections, and external interfaces can be helpful.>

Product Functions

<Summarize the major functions the product must perform or must let the user perform. Details will be provided in Section 3, so only a high-level summary (such as a bullet list) is needed here. Organize the functions to make them understandable to any reader of the SRS. A picture of the major groups of related requirements and how they relate, such as a top-level data flow diagram or object class diagram, is often effective.>

User Classes and Characteristics

<Identify the various user classes that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.>

Operating Environment

<Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.>

Design and Implementation Constraints

<Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).>

User Documentation

<List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.>

Assumptions and Dependencies

<List any assumed factors (as opposed to known facts) that could affect the requirements stated in the SRS. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).>

External Interface Requirements

User Interfaces

<Describe the logical characteristics of each interface between the software product and the users. This may include sample screen images, any GUI standards or product family style guides that are to be followed, screen layout constraints, standard buttons and functions (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, and so on. Define the software components for which a user interface is needed. Details of the user interface design should be documented in a separate user interface specification.>

Hardware Interfaces

<Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.>

Software Interfaces

<Describe the connections between this product and other specific software components (name and version), including databases, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Refer to documents that describe detailed application programming interface protocols. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.>

Communications Interfaces

<Describe the requirements associated with any communications functions required by this product, including e-mail, web browser, network server communications protocols, electronic forms, and so on. Define any pertinent message formatting. Identify any communication standards that will be used, such as FTP or HTTP. Specify any communication security or encryption issues, data transfer rates, and synchronization mechanisms.>

System Features

<This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product. You may prefer to organize this section by use case, mode of operation, user class, object class, functional hierarchy, or combinations of these, whatever makes the most logical sense for your product.>

System Feature 1

<Don't really say "System Feature 1." State the feature name in just a few words.>

4.1.1 Description and Priority

<Provide a short description of the feature and indicate whether it is of High, Medium, or Low priority. You could also include specific priority component ratings, such as

benefit, penalty, cost, and risk (each rated on a relative scale from a low of 1 to a high of 9).>

4.1.2 Stimulus/Response Sequences

<List the sequences of user actions and system responses that stimulate the behavior defined for this feature. These will correspond to the dialog elements associated with use cases.>

4.1.3 Functional Requirements

<Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature, or to execute the use case. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Use "TBD" as a placeholder to indicate when necessary information is not yet available.>

<Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.>

REQ-1:

REQ-2:

System Feature 2 (and so on)

Other Nonfunctional Requirements

Performance Requirements

<If there are performance requirements for the product under various circumstances, state them here and explain their rationale, to help the developers understand the intent and make suitable design choices. Specify the timing relationships for real time systems. Make such requirements as specific as possible. You may need to state performance requirements for individual functional requirements or features.>

Safety Requirements

<Specify those requirements that are concerned with possible loss, damage, or harm that could result from the use of the product. Define any safeguards or actions that must be taken, as well as actions that must be prevented. Refer to any external policies or regulations that state safety issues that affect the product's design or use. Define any safety certifications that must be satisfied.>

Security Requirements

<Specify any requirements regarding security or privacy issues surrounding use of the product or protection of the data used or created by the product. Define any user identity authentication requirements. Refer to any external policies or regulations containing security issues that affect the product. Define any security or privacy certifications that must be satisfied.>

Software Quality Attributes

<Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability. Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.>

Business Rules

<List any operating principles about the product, such as which individuals or roles can perform which functions under specific circumstances. These are not functional requirements in themselves, but they may imply certain functional requirements to enforce the rules.>

Other Requirements

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Appendix B: Analysis Models

<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>